



# Capstone Project



## CSA0242 – C Programming for Firmware

**Title of the Project:** Student's Mark Management system using C-Programming

**NAME:** K.NIKHIL  
**REG. NO. :** 192525307  
**Dept. of CSE**  
**SIMATS, Thandalam, Chennai.**

**NAME:** B.VENKATA  
REDDY  
**REG. NO. :** 192525362  
**Dept. of CSE**  
**SIMATS, Thandalam, C**  
**hennai.**

### Supervisors

**Dr. M. Kathiravan &**  
**Dr. A. Jagadeesan**  
**Computer Science & Engineering**  
**SIMATS, Thandalam, Chennai.**

# INTRODUCTION

---

- Organizes Student Data – It stores details like name, roll number, subjects, and marks in one place.
- Easy Calculation – It helps to calculate total marks, average, and grades quickly without mistakes.
- Saves Time – Teachers and students can find results faster compared to manual paper work.
- Improves Accuracy – Reduces human errors in recording and calculating marks.
- User-Friendly – Simple to use and understand for both teachers and students.

# NEED OF THE SYSTEM

---

- Retrieving or searching specific student data is very difficult in paper-based systems. student marks is time-consuming and inefficient.
- Teachers can quickly enter, update, and calculate results without spending extra time on manual calculations.
- There are high chances of errors in manual calculation of totals and averages.
- Teachers face problems in generating quick reports and performance analysis.
- An automated system improves accuracy, saves time, and ensures reliable results.

# LITERATURE REVIEW

| Year | Paper Title             | Problem Addressed                | Methodology                                              | Key Findings                             | Limitation                           |
|------|-------------------------|----------------------------------|----------------------------------------------------------|------------------------------------------|--------------------------------------|
| 2025 | Problem analysis        | Student manual management system | studied manual marks entry and result calculation issues | Effective standardized matching          | Identified inefficacy and time delay |
| 2024 | System design           | Large-scale resume–job retrieval | used c structures, functions and validation logic        | Structured and modular approach          | No GUI, only console                 |
| 2024 | LLMs for Recommendation | Large-scale resume–job retrieval | Contrastive dual encoders + FAISS ANN search             | Strong ranking and retrieval performance | Needs high-quality negatives         |



# PROBLEM DEFINITION

---

- Manual record-keeping issues – In traditional systems, student marks are recorded on paper or spreadsheets, which leads to errors, difficulty in updating, time consumption, and lack of security.
- Difficulty in accessing and analyzing results – Teachers and administrators face challenges in retrieving student performance quickly, generating reports, and providing accurate insights for decision-making

# OBJECTIVES

---

1. Efficient Record Management – To store, organize, and manage student marks systematically, reducing errors and paperwork.
2. Quick Access & Retrieval – To allow teachers and administrators to easily enter, update, and retrieve student performance data.
3. Performance Analysis – To generate reports, calculate grades, and analyze student progress for better decision-making.

# METHODOLOGY

---

## **Problem Identification**

- First, the issue of manual student marks management was analyzed.
- A computerized system was chosen to reduce errors and save time.

## **Requirement Analysis**

- The system should allow input of roll numbers, student names, and marks.
- The program should calculate total marks, average, and pass/fail status automatically.

## **System Design**

- Functions were used for validation, calculation, and display of reports.

# METHODOLOGY

---

## **Testing & Evaluation**

- Cases with valid/invalid names, roll numbers, and marks were checked.
- Output was verified to ensure correct totals, averages, and pass/fail status.

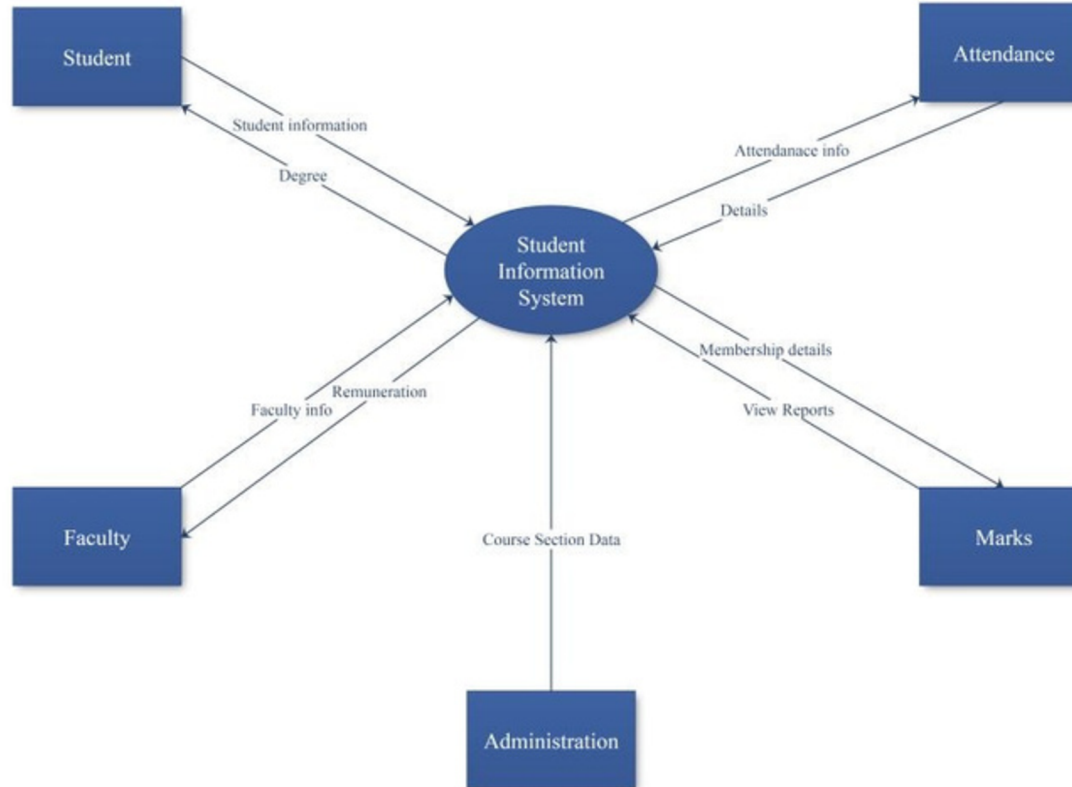
## **Documentation & Finalization**

- A user-friendly report was generated showing each student's details and results.
- Limitations and future scope (like adding file storage or GUI) were documents.

# METHODOLOGY

---

## Student Information system



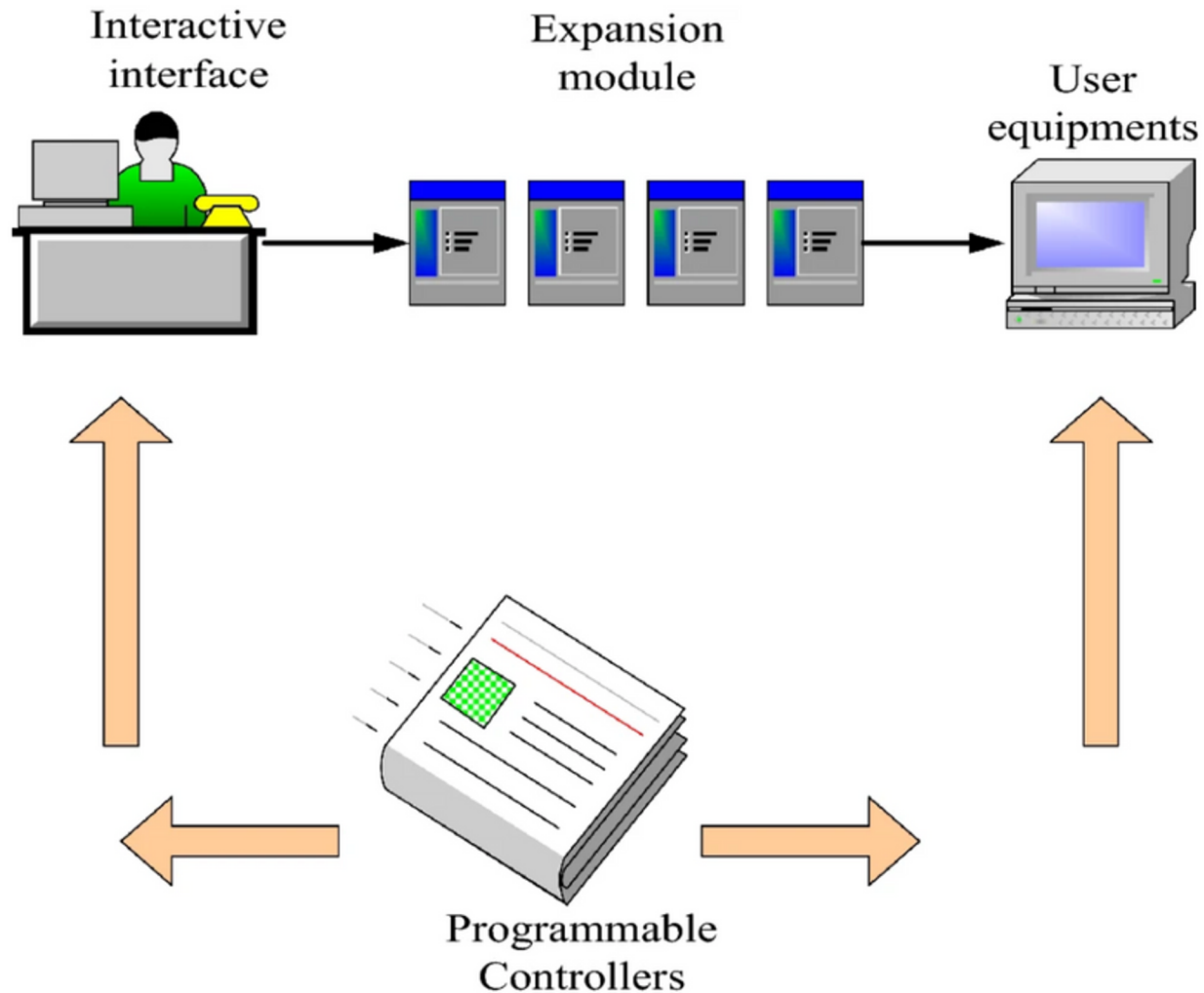
# IMPLEMENTATION

- **Homepage**



# IMPLEMENTATION

- Manual



# IMPLEMENTATION

---

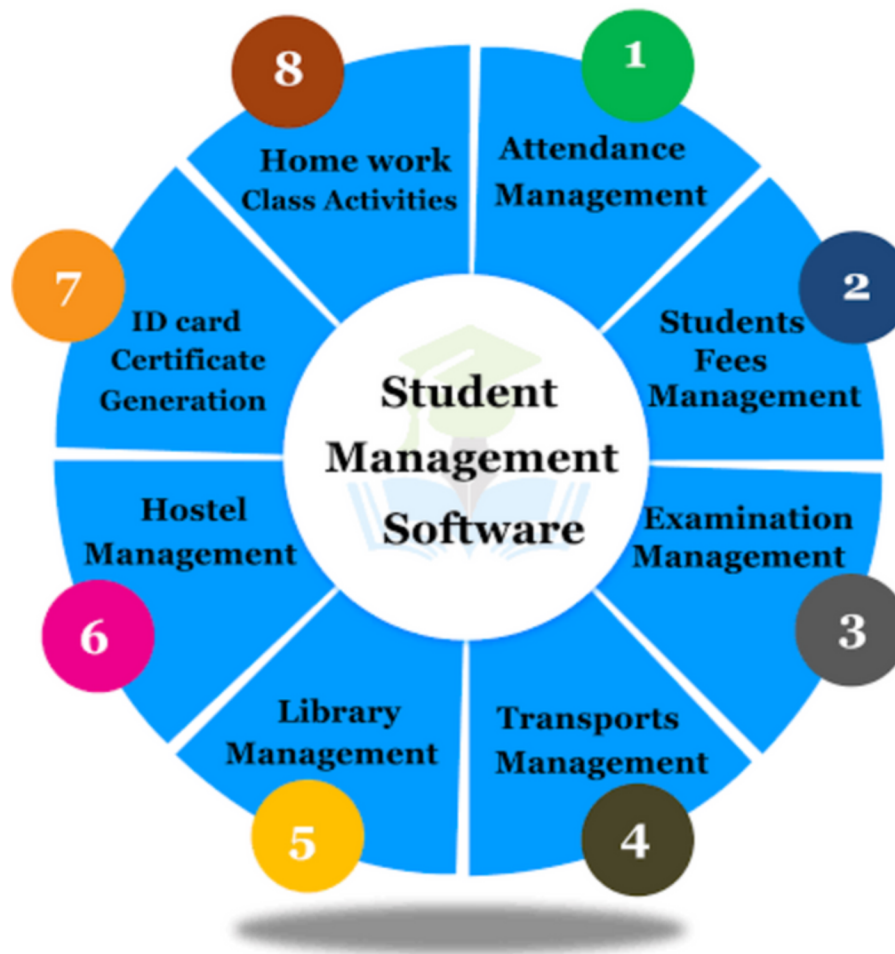
- **It saves time and works more efficiently**





# IMPLEMENTATION

- Overall review



# Coding

---

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

struct Student {
    int roll;
    char name[50];
    int marks[5];
    int total;
    float average;
    int pass;
    char failedSubjects[200];
};

int isValidName(char name[]) {
    for (int i = 0; i < strlen(name); i++) {
        if (!isalpha(name[i]) && name[i] != ' ') {
            return 0;
        }
    }
    return 1;
}

int main() {
    int n, i, j;

    printf("Enter number of students: ");
    while (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input! Enter a valid positive number of students: ");
    }
}
```

# Coding

---

```
while (getchar() != '\n');
    }

    struct Student s[n];

    for (i = 0; i < n; i++) {
        printf("\nEnter details of student %d\n", i + 1);

        do {
            printf("Roll Number: ");
            while (scanf("%d", &s[i].roll) != 1) {
                printf("Invalid input! Enter a valid Roll Number (only positive numbers): ");
                while (getchar() != '\n');
            }
            if (s[i].roll <= 0) {
                printf("Invalid! Roll number must be positive.\n");
            }
        } while (s[i].roll <= 0);

        do {
            printf("Name: ");
            scanf(" %[^\n]", s[i].name);
            if (!isValidName(s[i].name)) {
                printf("Invalid name! Please enter alphabets only.\n");
            }
        } while (!isValidName(s[i].name));
```

# Coding

---

```
s[i].total = 0;
s[i].pass = 1;
strcpy(s[i].failedSubjects, "");

for (j = 0; j < 5; j++) {
    do {
        printf("Enter marks of subject %d (0-100): ", j + 1);
        while (scanf("%d", &s[i].marks[j]) != 1) {
            printf("Invalid input! Enter integer marks only (0-100): ");
            while (getchar() != '\n');
        }
        if (s[i].marks[j] < 0 || s[i].marks[j] > 100) {
            printf("Marks must be between 0 and 100! Please re-enter.\n");
        }
    } while (s[i].marks[j] < 0 || s[i].marks[j] > 100);

    s[i].total += s[i].marks[j];

    if (s[i].marks[j] < 35) {
        s[i].pass = 0; // student failed
        char temp[50];
        sprintf(temp, "Subject%d ", j + 1);
        strcat(s[i].failedSubjects, temp);
    }
}
```

# Coding

---

```
s[i].average = s[i].total / 5.0;
    }

printf("\n----- Student Marks Report ----- \n");
printf("Roll\tName\t\tTotal\tAverage\tResult\n");

    for (i = 0; i < n; i++) {
printf("%d\t%-15s\t%d\t%.2f\t", s[i].roll, s[i].name, s[i].total, s[i].average);
        if (s[i].pass) {
            printf("PASS\n");
        } else {
            printf("FAIL (in %s)\n", s[i].failedSubjects);
        }
    }

    return 0;
}
```

# RESULT ANALYSIS

---

## Input :

```
Enter number of students: 2

Enter details of student 1
Roll Number: 01
Name: Nikhil
Enter marks of subject 1 (0-100): 95
Enter marks of subject 2 (0-100): 65
Enter marks of subject 3 (0-100): 88
Enter marks of subject 4 (0-100): 33
Enter marks of subject 5 (0-100): 35

Enter details of student 2
Roll Number: 02
Name: Venkata Reddy
Enter marks of subject 1 (0-100): 69
Enter marks of subject 2 (0-100): 49
Enter marks of subject 3 (0-100): 98
Enter marks of subject 4 (0-100): 88
Enter marks of subject 5 (0-100): 77
```

# RESULT ANALYSIS

---

**Output :**

```
----- Student Marks Report -----
Roll    Name          Total    Average Result
1    Nikhil          316 63.20    FAIL (in Subject4
    )
2    Venkata Reddy    381 76.20    PASS
```

# CONCLUSION

---

- The Student Marks Management System proves to be a simple yet effective solution for managing students' academic records.
- It eliminates errors associated with manual calculation and ensures data accuracy with proper validation techniques.
- This system can be extended into a full-fledged application with features such as database storage, grade allocation, and graphical analysis of performance.
- It provides accurate, fast, and transparent reports, and can be extended in the future with grading and database support.



# REFERENCES

---

1. E. Balagurusamy. Programming in ANSI C. Tata McGraw-Hill Education, 8th Edition, New Delhi, India, 2019.
2. Brian W. Kernighan, Dennis M. Ritchie. The C Programming Language. Prentice Hall, 2nd Edition, Englewood Cliffs, New Jersey, USA, 1988.
3. Yashavant Kanetkar. Let Us C. BPB Publications, 17th Edition, New Delhi, India, 2020.
4. Tutorialspoint. "C Programming Language Tutorial." Tutorialspoint.com. Retrieved from: <https://www.tutorialspoint.com/cprogramming/index.htm>
5. GeeksforGeeks. "C Programming Language – Basics, Functions, and Examples." GeeksforGeeks.org. Retrieved from: <https://www.geeksforgeeks.org/c-programming-language>
6. Research Articles. IEEE Xplore Digital Library – “Automated Student Information Management Systems” (IEEE, 2018).

---

**Thank you**